

```

import os
import matplotlib.pyplot as plt
from PIL import Image
import matplotlib.gridspec as gridspec
import numpy as np

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
# Added random_split to split the data

if torch.backends.mps.is_available():
    device = torch.device("mps")
elif torch.cuda.is_available():
    device = torch.device("cuda")
else:
    device = torch.device("cpu")
print("Using device:", device)

Using device: cuda

```

Problem 4.1: Organize the Data

Data should be organized to facilitate training, testing, and validating a model

```

# define the training and testing mechanisms to load in images as datasets
BATCH_SIZE = 10

train_transform_augmented = transforms.Compose([
    # Apply some random flips and rotation
    transforms.RandomResizedCrop(200, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(p = 0.7),
    transforms.RandomVerticalFlip(p = 0.7),
    transforms.RandomRotation(degrees = 25),

    # Randomly apply color jitters or random perspectives
    transforms.RandomApply([
        transforms.ColorJitter(brightness = 0.3, contrast = 0.3,
            saturation = 0.3, hue = 0.05),
        transforms.RandomPerspective(distortion_scale = 0.4, p = 1.0),
    ], p = 0.5),

    transforms.ToTensor(),
    transforms.Normalize(mean = [0.5, 0.5, 0.5], std = [0.5, 0.5,
0.5])
])

```

```

full_dataset_aug = datasets.ImageFolder("images", transform =
train_transform_augmented)

train_size_aug = int(0.8 * len(full_dataset_aug))
test_size_aug = len(full_dataset_aug) - train_size_aug

train_dataset_augmented, test_dataset_augmented =
random_split(full_dataset_aug, [train_size_aug, test_size_aug])
train_loader_augmented = DataLoader(train_dataset_augmented,
batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset_augmented,
batch_size=BATCH_SIZE, shuffle=False)

```

Problem 4.2: Construct a CNN

Implement a CNN in PyTorch with appropriate layers and a loss function

```

class ClassificationCNN(nn.Module):
    def __init__(self):
        super(ClassificationCNN, self).__init__()

        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(p=0.1),

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(p=0.15),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(p=0.2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(p=0.25)
        )

        conv_output_size = 128 * (12) * (12)
        self.fc_layers = nn.Sequential(
            nn.Flatten(),
            nn.Linear(conv_output_size, 256),
            nn.ReLU(),
            nn.Dropout(p=0.5),

```

```

        nn.Linear(256, 128),
        nn.ReLU(),
        nn.Dropout(p=0.4),
        nn.Linear(128, 8)
    )

    # define the forward step
    def forward(self, x):
        x = self.conv_layers(x)
        x = self.fc_layers(x)
        return x

```

Problem 4.3: Training the CNN

Implement a training loop to optimize the weights in the network

```

def compute_correct_labels(outputs, labels):
    correct = 0
    total = 0
    _, predicted_values = torch.max(outputs, 1)
    differences = predicted_values - labels
    for i in range(len(differences)):
        if differences[i]==0:
            correct +=1
        total += 1
    return(correct, total)

def training_loop(model, optimizer, NUM_EPOCHS, train_loader,
test_loader, printing=True):
    # make empty lists to keep track of the training and testing losses
    train_losses = []
    test_losses = []

    # loop through each epoch to run the training loop
    # and check the model with the training data
    # keep track of both sets of losses as your go
    for epoch in range(NUM_EPOCHS):

        # Run the training loop
        model.train()
        total_train_loss = 0.0
        total_train_correct = 0
        total_train_images = 0
        for train_inputs, train_labels in train_loader:
            train_inputs, train_labels = train_inputs.to(device),
train_labels.to(device)
            optimizer.zero_grad()
            outputs = model(train_inputs)

```

```

        train_correct, train_total =
compute_correct_labels(outputs, train_labels)
        total_train_correct += train_correct
        total_train_images += train_total
        loss = criterion(outputs, train_labels)
        loss.backward()
        optimizer.step()
        total_train_loss += loss.item()
    avg_train_loss = total_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    # Run the testing loop
    # this is essentially the same as the training loop but
    # without the optimizer and backward propagation
    model.eval()
    total_test_loss = 0.0
    total_test_correct = 0
    total_test_images = 0
    with torch.no_grad():
        for test_inputs, test_labels in test_loader:
            test_inputs, test_labels = test_inputs.to(device),
test_labels.to(device)
            outputs = model(test_inputs)
            test_correct, test_total =
compute_correct_labels(outputs, test_labels)
            total_test_correct += test_correct
            total_test_images += test_total
            loss = criterion(outputs, test_labels)
            total_test_loss += loss.item()
    avg_test_loss = total_test_loss / len(test_loader)
    test_losses.append(avg_test_loss)

    if printing:
        print(f"Epoch {epoch+1}/{NUM_EPOCHS}" + \
              f" - Train Loss: {avg_train_loss:.4f}, "+ \
              f"Train Correct:
{total_train_correct}/{total_train_images} "+ \
              f"- Test Loss: {avg_test_loss:.4f}, "+ \
              f"Test Correct:
{total_test_correct}/{total_test_images} ")

    return(train_losses, test_losses)

model = ClassificationCNN().to(device)

criterion = nn.CrossEntropyLoss()

# Adam optimizer for stochastic gradient descent
optimizer = optim.Adam(model.parameters(), lr=0.0005, weight_decay=1e-
4)

```

```
# define the number of epochs
```

```
NUM_EPOCHS = 200
```

```
# call the training loop function we defined above
```

```
train_losses, test_losses = training_loop(model, optimizer,  
NUM_EPOCHS, train_loader_augmented, test_loader, printing = True)
```

```
Epoch 1/200 - Train Loss: 2.0787, Train Correct: 80/607 - Test Loss:  
2.0509, Test Correct: 22/152
```

```
Epoch 2/200 - Train Loss: 2.0603, Train Correct: 90/607 - Test Loss:  
2.0425, Test Correct: 22/152
```

```
Epoch 3/200 - Train Loss: 2.0396, Train Correct: 93/607 - Test Loss:  
2.0446, Test Correct: 32/152
```

```
Epoch 4/200 - Train Loss: 2.0021, Train Correct: 115/607 - Test Loss:  
2.0060, Test Correct: 35/152
```

```
Epoch 5/200 - Train Loss: 1.9659, Train Correct: 117/607 - Test Loss:  
2.0141, Test Correct: 30/152
```

```
Epoch 6/200 - Train Loss: 1.9531, Train Correct: 145/607 - Test Loss:  
1.9583, Test Correct: 43/152
```

```
Epoch 7/200 - Train Loss: 1.9164, Train Correct: 138/607 - Test Loss:  
1.9064, Test Correct: 42/152
```

```
Epoch 8/200 - Train Loss: 1.8944, Train Correct: 139/607 - Test Loss:  
1.9081, Test Correct: 47/152
```

```
Epoch 9/200 - Train Loss: 1.9026, Train Correct: 134/607 - Test Loss:  
1.9812, Test Correct: 38/152
```

```
Epoch 10/200 - Train Loss: 1.8959, Train Correct: 141/607 - Test Loss:  
1.8738, Test Correct: 43/152
```

```
Epoch 11/200 - Train Loss: 1.8625, Train Correct: 156/607 - Test Loss:  
1.9351, Test Correct: 38/152
```

```
Epoch 12/200 - Train Loss: 1.9027, Train Correct: 144/607 - Test Loss:  
1.8409, Test Correct: 48/152
```

```
Epoch 13/200 - Train Loss: 1.8439, Train Correct: 146/607 - Test Loss:  
1.8763, Test Correct: 52/152
```

```
Epoch 14/200 - Train Loss: 1.7908, Train Correct: 187/607 - Test Loss:  
1.8214, Test Correct: 49/152
```

```
Epoch 15/200 - Train Loss: 1.7855, Train Correct: 175/607 - Test Loss:  
1.8427, Test Correct: 52/152
```

```
Epoch 16/200 - Train Loss: 1.7312, Train Correct: 188/607 - Test Loss:  
1.8039, Test Correct: 54/152
```

```
Epoch 17/200 - Train Loss: 1.7614, Train Correct: 198/607 - Test Loss:  
1.9028, Test Correct: 35/152
```

```
Epoch 18/200 - Train Loss: 1.7457, Train Correct: 170/607 - Test Loss:  
1.8278, Test Correct: 51/152
```

```
Epoch 19/200 - Train Loss: 1.7027, Train Correct: 212/607 - Test Loss:  
1.8032, Test Correct: 52/152
```

```
Epoch 20/200 - Train Loss: 1.7192, Train Correct: 216/607 - Test Loss:  
1.7279, Test Correct: 57/152
```

```
Epoch 21/200 - Train Loss: 1.7049, Train Correct: 210/607 - Test Loss:  
1.7209, Test Correct: 53/152
```

Epoch 22/200 - Train Loss: 1.6447, Train Correct: 206/607 - Test Loss: 1.7099, Test Correct: 59/152
Epoch 23/200 - Train Loss: 1.6416, Train Correct: 221/607 - Test Loss: 1.6698, Test Correct: 51/152
Epoch 24/200 - Train Loss: 1.6730, Train Correct: 230/607 - Test Loss: 1.6722, Test Correct: 58/152
Epoch 25/200 - Train Loss: 1.6037, Train Correct: 223/607 - Test Loss: 1.6609, Test Correct: 57/152
Epoch 26/200 - Train Loss: 1.5733, Train Correct: 242/607 - Test Loss: 1.6898, Test Correct: 53/152
Epoch 27/200 - Train Loss: 1.5557, Train Correct: 252/607 - Test Loss: 1.6440, Test Correct: 60/152
Epoch 28/200 - Train Loss: 1.5871, Train Correct: 228/607 - Test Loss: 1.6821, Test Correct: 57/152
Epoch 29/200 - Train Loss: 1.5595, Train Correct: 244/607 - Test Loss: 1.6583, Test Correct: 63/152
Epoch 30/200 - Train Loss: 1.5770, Train Correct: 250/607 - Test Loss: 1.7146, Test Correct: 48/152
Epoch 31/200 - Train Loss: 1.5557, Train Correct: 252/607 - Test Loss: 1.7238, Test Correct: 51/152
Epoch 32/200 - Train Loss: 1.5152, Train Correct: 245/607 - Test Loss: 1.6624, Test Correct: 61/152
Epoch 33/200 - Train Loss: 1.5107, Train Correct: 253/607 - Test Loss: 1.5969, Test Correct: 57/152
Epoch 34/200 - Train Loss: 1.4828, Train Correct: 267/607 - Test Loss: 1.5424, Test Correct: 67/152
Epoch 35/200 - Train Loss: 1.4678, Train Correct: 275/607 - Test Loss: 1.5866, Test Correct: 61/152
Epoch 36/200 - Train Loss: 1.4892, Train Correct: 262/607 - Test Loss: 1.5081, Test Correct: 72/152
Epoch 37/200 - Train Loss: 1.4322, Train Correct: 291/607 - Test Loss: 1.5678, Test Correct: 67/152
Epoch 38/200 - Train Loss: 1.4596, Train Correct: 267/607 - Test Loss: 1.5659, Test Correct: 61/152
Epoch 39/200 - Train Loss: 1.4003, Train Correct: 282/607 - Test Loss: 1.5240, Test Correct: 63/152
Epoch 40/200 - Train Loss: 1.4406, Train Correct: 276/607 - Test Loss: 1.5145, Test Correct: 60/152
Epoch 41/200 - Train Loss: 1.4372, Train Correct: 272/607 - Test Loss: 1.5292, Test Correct: 70/152
Epoch 42/200 - Train Loss: 1.3818, Train Correct: 288/607 - Test Loss: 1.4486, Test Correct: 72/152
Epoch 43/200 - Train Loss: 1.4267, Train Correct: 289/607 - Test Loss: 1.5402, Test Correct: 65/152
Epoch 44/200 - Train Loss: 1.3925, Train Correct: 296/607 - Test Loss: 1.5619, Test Correct: 66/152
Epoch 45/200 - Train Loss: 1.3523, Train Correct: 296/607 - Test Loss: 1.5027, Test Correct: 68/152
Epoch 46/200 - Train Loss: 1.4300, Train Correct: 290/607 - Test Loss:

1.4226, Test Correct: 70/152
Epoch 47/200 - Train Loss: 1.3494, Train Correct: 310/607 - Test Loss: 1.4553, Test Correct: 73/152
Epoch 48/200 - Train Loss: 1.3127, Train Correct: 306/607 - Test Loss: 1.4737, Test Correct: 70/152
Epoch 49/200 - Train Loss: 1.3614, Train Correct: 299/607 - Test Loss: 1.5095, Test Correct: 71/152
Epoch 50/200 - Train Loss: 1.4489, Train Correct: 293/607 - Test Loss: 1.5422, Test Correct: 59/152
Epoch 51/200 - Train Loss: 1.3695, Train Correct: 305/607 - Test Loss: 1.4211, Test Correct: 70/152
Epoch 52/200 - Train Loss: 1.3110, Train Correct: 307/607 - Test Loss: 1.4925, Test Correct: 67/152
Epoch 53/200 - Train Loss: 1.3282, Train Correct: 308/607 - Test Loss: 1.4203, Test Correct: 73/152
Epoch 54/200 - Train Loss: 1.3436, Train Correct: 296/607 - Test Loss: 1.3880, Test Correct: 76/152
Epoch 55/200 - Train Loss: 1.2971, Train Correct: 308/607 - Test Loss: 1.4641, Test Correct: 71/152
Epoch 56/200 - Train Loss: 1.3381, Train Correct: 298/607 - Test Loss: 1.3981, Test Correct: 74/152
Epoch 57/200 - Train Loss: 1.3207, Train Correct: 290/607 - Test Loss: 1.3473, Test Correct: 75/152
Epoch 58/200 - Train Loss: 1.3174, Train Correct: 313/607 - Test Loss: 1.3972, Test Correct: 71/152
Epoch 59/200 - Train Loss: 1.3233, Train Correct: 304/607 - Test Loss: 1.5092, Test Correct: 71/152
Epoch 60/200 - Train Loss: 1.2521, Train Correct: 322/607 - Test Loss: 1.4777, Test Correct: 62/152
Epoch 61/200 - Train Loss: 1.2808, Train Correct: 313/607 - Test Loss: 1.3771, Test Correct: 78/152
Epoch 62/200 - Train Loss: 1.2668, Train Correct: 331/607 - Test Loss: 1.3110, Test Correct: 80/152
Epoch 63/200 - Train Loss: 1.2594, Train Correct: 320/607 - Test Loss: 1.2944, Test Correct: 80/152
Epoch 64/200 - Train Loss: 1.2039, Train Correct: 325/607 - Test Loss: 1.4410, Test Correct: 77/152
Epoch 65/200 - Train Loss: 1.2150, Train Correct: 333/607 - Test Loss: 1.3758, Test Correct: 75/152
Epoch 66/200 - Train Loss: 1.2118, Train Correct: 332/607 - Test Loss: 1.3623, Test Correct: 77/152
Epoch 67/200 - Train Loss: 1.1812, Train Correct: 352/607 - Test Loss: 1.3035, Test Correct: 75/152
Epoch 68/200 - Train Loss: 1.2037, Train Correct: 331/607 - Test Loss: 1.2604, Test Correct: 83/152
Epoch 69/200 - Train Loss: 1.2105, Train Correct: 336/607 - Test Loss: 1.3265, Test Correct: 86/152
Epoch 70/200 - Train Loss: 1.1767, Train Correct: 329/607 - Test Loss: 1.3485, Test Correct: 75/152

Epoch 71/200 - Train Loss: 1.1666, Train Correct: 338/607 - Test Loss: 1.3066, Test Correct: 83/152
Epoch 72/200 - Train Loss: 1.1365, Train Correct: 351/607 - Test Loss: 1.2807, Test Correct: 73/152
Epoch 73/200 - Train Loss: 1.1718, Train Correct: 343/607 - Test Loss: 1.4321, Test Correct: 71/152
Epoch 74/200 - Train Loss: 1.2211, Train Correct: 320/607 - Test Loss: 1.2069, Test Correct: 89/152
Epoch 75/200 - Train Loss: 1.1265, Train Correct: 347/607 - Test Loss: 1.3531, Test Correct: 80/152
Epoch 76/200 - Train Loss: 1.2094, Train Correct: 345/607 - Test Loss: 1.2689, Test Correct: 86/152
Epoch 77/200 - Train Loss: 1.1315, Train Correct: 350/607 - Test Loss: 1.2226, Test Correct: 86/152
Epoch 78/200 - Train Loss: 1.1828, Train Correct: 342/607 - Test Loss: 1.2977, Test Correct: 82/152
Epoch 79/200 - Train Loss: 1.1740, Train Correct: 352/607 - Test Loss: 1.2258, Test Correct: 92/152
Epoch 80/200 - Train Loss: 1.1268, Train Correct: 350/607 - Test Loss: 1.2434, Test Correct: 80/152
Epoch 81/200 - Train Loss: 1.1328, Train Correct: 327/607 - Test Loss: 1.1861, Test Correct: 94/152
Epoch 82/200 - Train Loss: 1.1695, Train Correct: 352/607 - Test Loss: 1.2785, Test Correct: 79/152
Epoch 83/200 - Train Loss: 1.0929, Train Correct: 368/607 - Test Loss: 1.1579, Test Correct: 83/152
Epoch 84/200 - Train Loss: 1.0733, Train Correct: 369/607 - Test Loss: 1.2605, Test Correct: 77/152
Epoch 85/200 - Train Loss: 1.0774, Train Correct: 352/607 - Test Loss: 1.2614, Test Correct: 76/152
Epoch 86/200 - Train Loss: 1.0800, Train Correct: 363/607 - Test Loss: 1.2381, Test Correct: 83/152
Epoch 87/200 - Train Loss: 1.0991, Train Correct: 351/607 - Test Loss: 1.3072, Test Correct: 82/152
Epoch 88/200 - Train Loss: 1.0900, Train Correct: 371/607 - Test Loss: 1.3054, Test Correct: 83/152
Epoch 89/200 - Train Loss: 1.0339, Train Correct: 381/607 - Test Loss: 1.2197, Test Correct: 81/152
Epoch 90/200 - Train Loss: 1.0767, Train Correct: 358/607 - Test Loss: 1.0949, Test Correct: 92/152
Epoch 91/200 - Train Loss: 1.0954, Train Correct: 363/607 - Test Loss: 1.3099, Test Correct: 82/152
Epoch 92/200 - Train Loss: 1.1090, Train Correct: 350/607 - Test Loss: 1.1638, Test Correct: 92/152
Epoch 93/200 - Train Loss: 1.1002, Train Correct: 363/607 - Test Loss: 1.1651, Test Correct: 87/152
Epoch 94/200 - Train Loss: 1.1190, Train Correct: 355/607 - Test Loss: 1.2033, Test Correct: 86/152
Epoch 95/200 - Train Loss: 1.0729, Train Correct: 374/607 - Test Loss:

1.1858, Test Correct: 81/152
Epoch 96/200 - Train Loss: 1.0613, Train Correct: 377/607 - Test Loss: 1.3397, Test Correct: 76/152
Epoch 97/200 - Train Loss: 1.0896, Train Correct: 349/607 - Test Loss: 1.1835, Test Correct: 77/152
Epoch 98/200 - Train Loss: 0.9948, Train Correct: 380/607 - Test Loss: 1.2954, Test Correct: 89/152
Epoch 99/200 - Train Loss: 1.0844, Train Correct: 358/607 - Test Loss: 1.2181, Test Correct: 86/152
Epoch 100/200 - Train Loss: 1.0989, Train Correct: 361/607 - Test Loss: 1.1915, Test Correct: 86/152
Epoch 101/200 - Train Loss: 1.1128, Train Correct: 365/607 - Test Loss: 1.1406, Test Correct: 87/152
Epoch 102/200 - Train Loss: 1.0603, Train Correct: 369/607 - Test Loss: 1.2529, Test Correct: 88/152
Epoch 103/200 - Train Loss: 0.9932, Train Correct: 382/607 - Test Loss: 1.1168, Test Correct: 87/152
Epoch 104/200 - Train Loss: 1.0224, Train Correct: 373/607 - Test Loss: 1.1956, Test Correct: 90/152
Epoch 105/200 - Train Loss: 1.0026, Train Correct: 391/607 - Test Loss: 1.2787, Test Correct: 87/152
Epoch 106/200 - Train Loss: 0.9958, Train Correct: 389/607 - Test Loss: 1.2156, Test Correct: 83/152
Epoch 107/200 - Train Loss: 0.9896, Train Correct: 393/607 - Test Loss: 1.2138, Test Correct: 88/152
Epoch 108/200 - Train Loss: 1.0159, Train Correct: 380/607 - Test Loss: 1.1453, Test Correct: 82/152
Epoch 109/200 - Train Loss: 0.9853, Train Correct: 379/607 - Test Loss: 1.2041, Test Correct: 81/152
Epoch 110/200 - Train Loss: 1.0128, Train Correct: 392/607 - Test Loss: 1.1778, Test Correct: 86/152
Epoch 111/200 - Train Loss: 0.9791, Train Correct: 385/607 - Test Loss: 1.2830, Test Correct: 75/152
Epoch 112/200 - Train Loss: 0.9811, Train Correct: 391/607 - Test Loss: 1.0687, Test Correct: 92/152
Epoch 113/200 - Train Loss: 0.9479, Train Correct: 394/607 - Test Loss: 1.1035, Test Correct: 91/152
Epoch 114/200 - Train Loss: 1.0076, Train Correct: 390/607 - Test Loss: 1.1563, Test Correct: 93/152
Epoch 115/200 - Train Loss: 1.0104, Train Correct: 387/607 - Test Loss: 1.0251, Test Correct: 96/152
Epoch 116/200 - Train Loss: 0.9227, Train Correct: 394/607 - Test Loss: 1.0915, Test Correct: 92/152
Epoch 117/200 - Train Loss: 0.9147, Train Correct: 392/607 - Test Loss: 1.0584, Test Correct: 92/152
Epoch 118/200 - Train Loss: 0.9739, Train Correct: 401/607 - Test Loss: 1.1132, Test Correct: 94/152
Epoch 119/200 - Train Loss: 0.9465, Train Correct: 391/607 - Test Loss: 1.0271, Test Correct: 91/152

Epoch 120/200 - Train Loss: 0.9285, Train Correct: 402/607 - Test Loss: 1.0437, Test Correct: 93/152
Epoch 121/200 - Train Loss: 1.0022, Train Correct: 374/607 - Test Loss: 1.3143, Test Correct: 82/152
Epoch 122/200 - Train Loss: 1.0236, Train Correct: 368/607 - Test Loss: 1.1027, Test Correct: 92/152
Epoch 123/200 - Train Loss: 0.9155, Train Correct: 403/607 - Test Loss: 1.2694, Test Correct: 85/152
Epoch 124/200 - Train Loss: 0.9993, Train Correct: 374/607 - Test Loss: 1.1435, Test Correct: 80/152
Epoch 125/200 - Train Loss: 0.8839, Train Correct: 411/607 - Test Loss: 1.0406, Test Correct: 98/152
Epoch 126/200 - Train Loss: 0.9494, Train Correct: 404/607 - Test Loss: 1.1484, Test Correct: 87/152
Epoch 127/200 - Train Loss: 0.9125, Train Correct: 400/607 - Test Loss: 1.1235, Test Correct: 93/152
Epoch 128/200 - Train Loss: 0.9145, Train Correct: 400/607 - Test Loss: 1.2121, Test Correct: 92/152
Epoch 129/200 - Train Loss: 0.9093, Train Correct: 418/607 - Test Loss: 1.1219, Test Correct: 85/152
Epoch 130/200 - Train Loss: 0.9447, Train Correct: 411/607 - Test Loss: 1.0930, Test Correct: 87/152
Epoch 131/200 - Train Loss: 0.9669, Train Correct: 392/607 - Test Loss: 1.1893, Test Correct: 93/152
Epoch 132/200 - Train Loss: 0.9282, Train Correct: 414/607 - Test Loss: 1.0560, Test Correct: 98/152
Epoch 133/200 - Train Loss: 0.8889, Train Correct: 411/607 - Test Loss: 1.1037, Test Correct: 93/152
Epoch 134/200 - Train Loss: 0.8593, Train Correct: 414/607 - Test Loss: 1.0011, Test Correct: 103/152
Epoch 135/200 - Train Loss: 0.9702, Train Correct: 404/607 - Test Loss: 1.0464, Test Correct: 96/152
Epoch 136/200 - Train Loss: 0.8771, Train Correct: 408/607 - Test Loss: 1.0975, Test Correct: 99/152
Epoch 137/200 - Train Loss: 0.8975, Train Correct: 399/607 - Test Loss: 1.0528, Test Correct: 93/152
Epoch 138/200 - Train Loss: 0.9008, Train Correct: 403/607 - Test Loss: 1.0619, Test Correct: 96/152
Epoch 139/200 - Train Loss: 0.9081, Train Correct: 407/607 - Test Loss: 1.1197, Test Correct: 95/152
Epoch 140/200 - Train Loss: 0.9125, Train Correct: 402/607 - Test Loss: 1.1520, Test Correct: 89/152
Epoch 141/200 - Train Loss: 0.9022, Train Correct: 418/607 - Test Loss: 1.2194, Test Correct: 97/152
Epoch 142/200 - Train Loss: 0.9169, Train Correct: 399/607 - Test Loss: 1.0660, Test Correct: 92/152
Epoch 143/200 - Train Loss: 0.9502, Train Correct: 410/607 - Test Loss: 1.0064, Test Correct: 94/152
Epoch 144/200 - Train Loss: 0.8039, Train Correct: 432/607 - Test

Loss: 1.1271, Test Correct: 91/152
Epoch 145/200 - Train Loss: 0.9897, Train Correct: 399/607 - Test
Loss: 1.1134, Test Correct: 92/152
Epoch 146/200 - Train Loss: 0.8808, Train Correct: 421/607 - Test
Loss: 1.0772, Test Correct: 90/152
Epoch 147/200 - Train Loss: 0.8407, Train Correct: 416/607 - Test
Loss: 1.1958, Test Correct: 96/152
Epoch 148/200 - Train Loss: 0.7889, Train Correct: 440/607 - Test
Loss: 1.0825, Test Correct: 97/152
Epoch 149/200 - Train Loss: 0.8509, Train Correct: 430/607 - Test
Loss: 1.0819, Test Correct: 91/152
Epoch 150/200 - Train Loss: 0.8443, Train Correct: 425/607 - Test
Loss: 1.0832, Test Correct: 100/152
Epoch 151/200 - Train Loss: 0.8832, Train Correct: 423/607 - Test
Loss: 1.1348, Test Correct: 91/152
Epoch 152/200 - Train Loss: 0.8066, Train Correct: 427/607 - Test
Loss: 1.1749, Test Correct: 96/152
Epoch 153/200 - Train Loss: 0.8885, Train Correct: 414/607 - Test
Loss: 1.1317, Test Correct: 96/152
Epoch 154/200 - Train Loss: 0.8289, Train Correct: 437/607 - Test
Loss: 1.1711, Test Correct: 92/152
Epoch 155/200 - Train Loss: 0.8510, Train Correct: 430/607 - Test
Loss: 1.1938, Test Correct: 86/152
Epoch 156/200 - Train Loss: 0.9098, Train Correct: 410/607 - Test
Loss: 1.1292, Test Correct: 88/152
Epoch 157/200 - Train Loss: 0.8081, Train Correct: 426/607 - Test
Loss: 1.1528, Test Correct: 88/152
Epoch 158/200 - Train Loss: 0.9019, Train Correct: 420/607 - Test
Loss: 1.1572, Test Correct: 90/152
Epoch 159/200 - Train Loss: 0.8603, Train Correct: 426/607 - Test
Loss: 1.1384, Test Correct: 89/152
Epoch 160/200 - Train Loss: 0.8954, Train Correct: 401/607 - Test
Loss: 1.0436, Test Correct: 100/152
Epoch 161/200 - Train Loss: 0.8177, Train Correct: 426/607 - Test
Loss: 1.0938, Test Correct: 97/152
Epoch 162/200 - Train Loss: 0.8360, Train Correct: 420/607 - Test
Loss: 1.1266, Test Correct: 89/152
Epoch 163/200 - Train Loss: 0.7549, Train Correct: 436/607 - Test
Loss: 1.0704, Test Correct: 92/152
Epoch 164/200 - Train Loss: 0.8431, Train Correct: 438/607 - Test
Loss: 1.1756, Test Correct: 85/152
Epoch 165/200 - Train Loss: 0.8278, Train Correct: 418/607 - Test
Loss: 1.2220, Test Correct: 96/152
Epoch 166/200 - Train Loss: 0.8296, Train Correct: 427/607 - Test
Loss: 1.0933, Test Correct: 87/152
Epoch 167/200 - Train Loss: 0.8579, Train Correct: 422/607 - Test
Loss: 1.0717, Test Correct: 98/152
Epoch 168/200 - Train Loss: 0.8308, Train Correct: 431/607 - Test
Loss: 1.1321, Test Correct: 94/152

Epoch 169/200 - Train Loss: 0.8547, Train Correct: 415/607 - Test Loss: 1.0049, Test Correct: 104/152
Epoch 170/200 - Train Loss: 0.7808, Train Correct: 439/607 - Test Loss: 1.0024, Test Correct: 98/152
Epoch 171/200 - Train Loss: 0.8097, Train Correct: 434/607 - Test Loss: 1.1530, Test Correct: 91/152
Epoch 172/200 - Train Loss: 0.7538, Train Correct: 452/607 - Test Loss: 1.1501, Test Correct: 93/152
Epoch 173/200 - Train Loss: 0.8222, Train Correct: 420/607 - Test Loss: 1.2238, Test Correct: 93/152
Epoch 174/200 - Train Loss: 0.8135, Train Correct: 413/607 - Test Loss: 1.1557, Test Correct: 98/152
Epoch 175/200 - Train Loss: 0.7826, Train Correct: 437/607 - Test Loss: 1.1673, Test Correct: 98/152
Epoch 176/200 - Train Loss: 0.7148, Train Correct: 452/607 - Test Loss: 1.0944, Test Correct: 91/152
Epoch 177/200 - Train Loss: 0.8249, Train Correct: 430/607 - Test Loss: 0.9938, Test Correct: 91/152
Epoch 178/200 - Train Loss: 0.7827, Train Correct: 450/607 - Test Loss: 1.1077, Test Correct: 94/152
Epoch 179/200 - Train Loss: 0.7869, Train Correct: 437/607 - Test Loss: 1.0483, Test Correct: 98/152
Epoch 180/200 - Train Loss: 0.7500, Train Correct: 443/607 - Test Loss: 1.1604, Test Correct: 92/152
Epoch 181/200 - Train Loss: 0.7734, Train Correct: 431/607 - Test Loss: 1.2150, Test Correct: 92/152
Epoch 182/200 - Train Loss: 0.7919, Train Correct: 448/607 - Test Loss: 1.1099, Test Correct: 98/152
Epoch 183/200 - Train Loss: 0.7763, Train Correct: 440/607 - Test Loss: 0.9594, Test Correct: 92/152
Epoch 184/200 - Train Loss: 0.6985, Train Correct: 458/607 - Test Loss: 1.1626, Test Correct: 91/152
Epoch 185/200 - Train Loss: 0.7595, Train Correct: 456/607 - Test Loss: 1.1509, Test Correct: 95/152
Epoch 186/200 - Train Loss: 0.7176, Train Correct: 454/607 - Test Loss: 1.0133, Test Correct: 100/152
Epoch 187/200 - Train Loss: 0.7874, Train Correct: 443/607 - Test Loss: 1.2133, Test Correct: 89/152
Epoch 188/200 - Train Loss: 0.7531, Train Correct: 442/607 - Test Loss: 1.1630, Test Correct: 95/152
Epoch 189/200 - Train Loss: 0.7741, Train Correct: 447/607 - Test Loss: 1.1399, Test Correct: 100/152
Epoch 190/200 - Train Loss: 0.7927, Train Correct: 438/607 - Test Loss: 1.0057, Test Correct: 101/152
Epoch 191/200 - Train Loss: 0.7279, Train Correct: 444/607 - Test Loss: 1.0068, Test Correct: 101/152
Epoch 192/200 - Train Loss: 0.7096, Train Correct: 441/607 - Test Loss: 1.1922, Test Correct: 94/152
Epoch 193/200 - Train Loss: 0.7117, Train Correct: 459/607 - Test

```
Loss: 1.0565, Test Correct: 100/152
Epoch 194/200 - Train Loss: 0.7075, Train Correct: 451/607 - Test
Loss: 1.0830, Test Correct: 101/152
Epoch 195/200 - Train Loss: 0.7440, Train Correct: 446/607 - Test
Loss: 1.3368, Test Correct: 89/152
Epoch 196/200 - Train Loss: 0.7938, Train Correct: 447/607 - Test
Loss: 1.2417, Test Correct: 95/152
Epoch 197/200 - Train Loss: 0.7445, Train Correct: 449/607 - Test
Loss: 1.0738, Test Correct: 97/152
Epoch 198/200 - Train Loss: 0.7071, Train Correct: 454/607 - Test
Loss: 1.0380, Test Correct: 93/152
Epoch 199/200 - Train Loss: 0.7215, Train Correct: 446/607 - Test
Loss: 1.2099, Test Correct: 93/152
Epoch 200/200 - Train Loss: 0.7868, Train Correct: 453/607 - Test
Loss: 1.1245, Test Correct: 93/152
```

Problem 4.4: Assess the results of the CNN training

Plot model losses and compute other metrics for model performance. Based on performance, modify the model architecture and training to test different implementations of the model and improve the model results. Be sure to visualize some of the images and their true vs predicted labels in addition to the model metrics.

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, NUM_EPOCHS + 1), train_losses, 'd-', label='Training
Loss')
plt.plot(range(1, NUM_EPOCHS + 1), test_losses, 'd-', label='Test
Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Testing Loss Over Epochs')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
# define the species for each category
species =
['barn_owl', 'brandts_cormorant', 'california_gull', 'great_blue_heron', '
great_egret', 'osprey', 'pigeon_guillemot', 'turkey_vulture']

# Set model to evaluation mode
model.eval()

# make a figure object
fig = plt.figure(figsize=(12, 12))
gs = gridspec.GridSpec(5, len(species))

# loop through each species (column)
for s in range(len(species)):

    # loop through the image files
    file_list = os.listdir(os.path.join('images', species[s]))[:5]
    for file_count, file_name in enumerate(file_list):

        # load the image
        image = Image.open(os.path.join('images', species[s],
file_name)).convert('RGB')
        input_tensor =
train_transform_augmented(image).unsqueeze(0).to(device) # Add batch
dimension

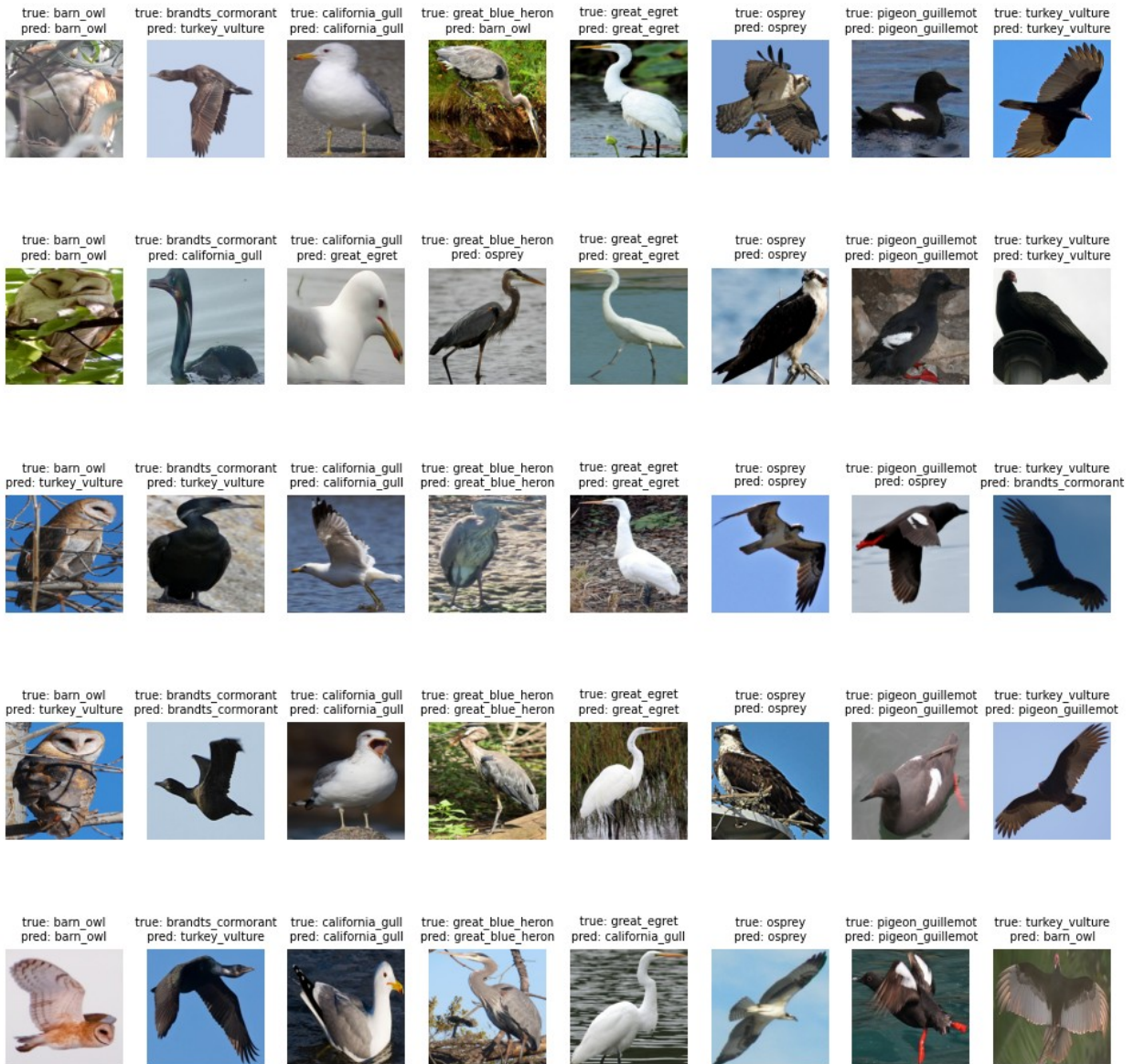
        # get the predicted class
        with torch.no_grad():
            output = model(input_tensor)
            _, predicted = torch.max(output, 1)
            predicted_class = species[predicted.item()]
```

```
    true_class = species[s]

    # add the image to the plot with the prediction
    ax = fig.add_subplot(gs[file_count, s])
    ax.imshow(image)
    ax.axis('off')
    ax.set_title(f"true: {true_class}\npred: {predicted_class}",
fontsize=7)
    file_count +=1

plt.suptitle('Predicted Labels of All Images in the Validation Dataset
after Training')
plt.show()
```

Predicted Labels of All Images in the Validation Dataset after Training



Problem 4.5: Describe the training process and the performance of the final model

Write a paragraph describing your approach to constructing your CNN. Explain some of the issues that arose in building your network and some approaches you took to try to alleviate these issues. Comment on what your model may be learning and why some of mislabeled images may not be classified correctly by the model.

I went with a CNN similar to our one in class, using a dropout and custom transformations to augment the images to prevent overfit.

My first issue was that the images weren't automatically split like in the lecture. For this I used a random split function afforded by Pytorch and used it to split the images. It splits the entire dataset rather than stratifying by category but it should be fine since each category is roughly similar sizes. After going bug hunting for a bit, I successfully implemented and ran the model.

The first run wasn't very good. My test loss started going up and up while my training loss decreased smoothly, pointing to a lot of overfitting. I decided to approach this by:

- Adding a random resized crop to the transform
- Adjusting some of the transform values as well
- Added more dropouts
- Adjusted the intensity of those dropouts
- Halved the learning rate.
- Reduced model capacity
- Added weight decay

After doing these, my test and training losses slowly but surely got pretty close. After this, I aimed to adjust the model itself to see if that would help the loss in general (as well as fix any overfit I could).

For my final act, I added 2 more layers with incrementally increasing dropout and adjusted the model size again. This didn't do much so I just resigned that this was as good as I was going to get it considering the sample sizes and samples. Speaking of the samples...

I think the samples are just tough. Lots of black birds being confused with each other. An especially dark image of a great blue heron is now an osprey, an image of a pigeon guillemot without its head shape clearly defined is an osprey as well. Sometimes barn owls are mistaken for turkey vultures. I think a lot of it is based on color, and maybe background. I bet a lot of california gull photos have ocean in the background so now once you put a brandts cormorant or great egret in the water the model thinks it's a gull.